
Reliable Transfer on Lossy, High-Latency Links: Round-Based NAK Retransmission over Paced UDP

Huacong Tang
University of Southern California

Xiao Yang
University of Southern California

Jiachen Lu
University of Southern California

 [Code](#)  [Demo Video](#)

Abstract

TCP prioritizes reliability and generality: its slow start, acknowledgment, and loss-as-congestion backoff are designed for unknown, shared networks. On dedicated point-to-point links, bandwidth, latency, and transfer size are known, and packet loss stems from the link rather than congestion. There, these mechanisms become the bottleneck: at a few percent random loss, TCP throughput collapses. We present a UDP-based reliable transport protocol for this setting. It removes congestion control and slow start and sends at a constant rate; loss never triggers slowdown. Reliability is instead guaranteed by a receiver-side bitmap with round-based batched NAKs that retransmit only missing packets. On a testbed with 100 Mbps links, 200 ms RTT, and 20% injected loss, the protocol delivers 1 GiB transfers at roughly $3000\times$ the throughput of TCP-based scp, requires no tuning across loss rates, and passes end-to-end integrity checks. These results suggest that on controlled links, abandoning probing mechanisms in favor of structural reliability guarantees is effective for high-loss transfer.

1 Problem and headline results

Move a 1 GiB file, one way, with zero errors, as fast as possible, over three impaired link configurations (Table 1). Reliability is a pass/fail gate proven by matching MD5 digests on both ends; speed is the competitive metric, with a 20 Mbps minimum (7.2 min for 1 GiB).

Figure 1 shows why the naive choices fail and where the opportunity is: over these same links, stock TCP (iperf) nearly vanishes—93 kbps under Case 2’s 20% random loss and only 15 Mbps under Case 3’s 200 ms RTT—while UDP delivers 78–97 Mbps regardless of loss or latency. TCP leaves the link almost empty; the UDP bars mark the ceiling our protocol targets.

Testbed. All experiments ran on three AWS EC2 instances (c7i-flex.large, Ubuntu 24.04) in one VPC — client 10.200.1.83, router, and server 10.200.2.48 on two subnets (10.200.1.0/24, 10.200.2.0/24), with every client–server packet forwarded through the router. Client and server egress is rate-limited to 100 Mbps (tbf); the router applies each case’s rate limit plus netem delay/loss on both its interfaces, exactly as the lab specifies; the MTU is 1500 or 9001 on all interfaces per experiment.

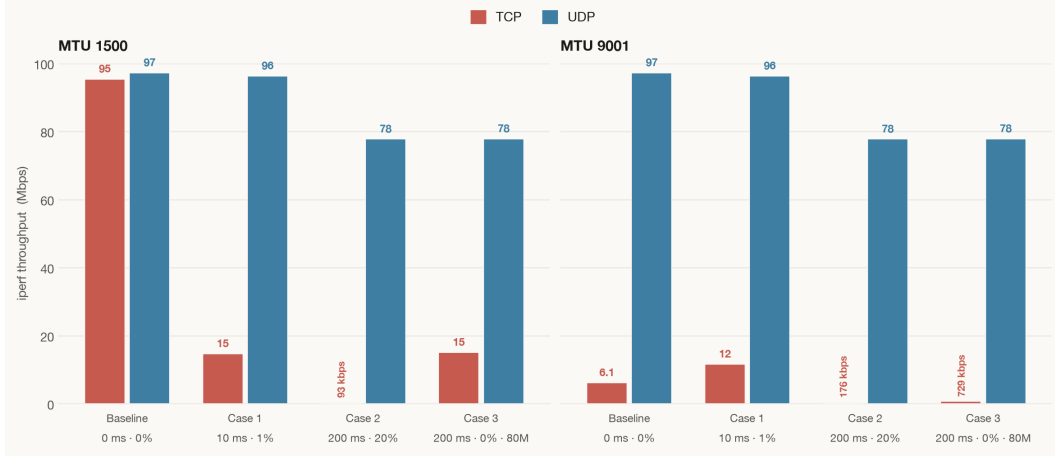


Figure 1: UDP ignores loss and RTT, so it barely changes across cases; TCP does not—Case 2 is loss-limited, Case 3 is window-limited by the high RTT.

Table 1: Headline results: 1 GiB one-way, MTU 1500, 3 repetitions per case, MD5 verified on both ends in every run. All cases clear the 20 Mbps competition threshold by 3.4–4.8 $\times$; speedup over scp is 6.1 $\times$ (Case 1) and 3.1 $\times$ (Case 3), and Case 2 completes where scp cannot. Loss is one-way data-path loss injected by netem at the router; timing is sender first-bit to receiver last-bit with measured clock-offset correction.

Case	Link (rate / RTT / loss)	scp (TCP baseline)	Ours, med. [min–max]	Goodput	Shannon $T_{\min}$	Gap
1	100 Mbps / 10 ms / 1%	548 s (15.7 Mbps)	90.0 s [90.0–90.0]	95.4 Mbps	88.9 s	+1.2%
2	100 Mbps / 200 ms / 20%	≈ 4 d (23 kbps)	124.8 s [124.7–125.3]	68.8 Mbps	110.0 s	+13%
3	80 Mbps / 200 ms / 0%	343 s (25.0 Mbps)	111.4 s [111.3–111.4]	77.1 Mbps	110.0 s	+1.2%

2 Design by deletion: a fully known, dedicated link

This is not the open Internet. The link is point-to-point, uncontended, and every parameter (rate, RTT, loss probability) is known in advance; loss is injected at random by netem, so it does not signal congestion. Each known property removes a mechanism that classical transfer tools pay for (Table 2). TCP’s collapse in Table 1 and Figure 1 comes from treating random loss as congestion; we are free to ignore it.

Table 2: Design by deletion.

Known property	Mechanism removed
Dedicated link, no competing flows	Congestion control (AIMD / BBR)
Line rate known and fixed	Slow start, probing: pace at $\approx 0.99R$ from $t = 0$
File size known at start	Receiver preallocates; missing = bitmap complement
Loss random, not congestive	Backoff: loss never reduces the sending rate

3 Protocol

Figure 2 shows one complete transfer, and Figure 3 gives the flow chart of the algorithm on each side. The data plane is UDP paced at the known line rate; the control plane is a per-round missing-sequence list (NAK); TCP appears only at the edges of the session.

Data structures. Three in total. The sender holds an mmap of the source file, so blocks are sent directly from the page cache with no per-block copies. The receiver holds the preallocated output file and a bitmap of N bits (~ 90 KB for 1 GiB) marking which blocks have arrived. Each round’s NAK payload is the missing-sequence list, i.e. the complement of the bitmap.

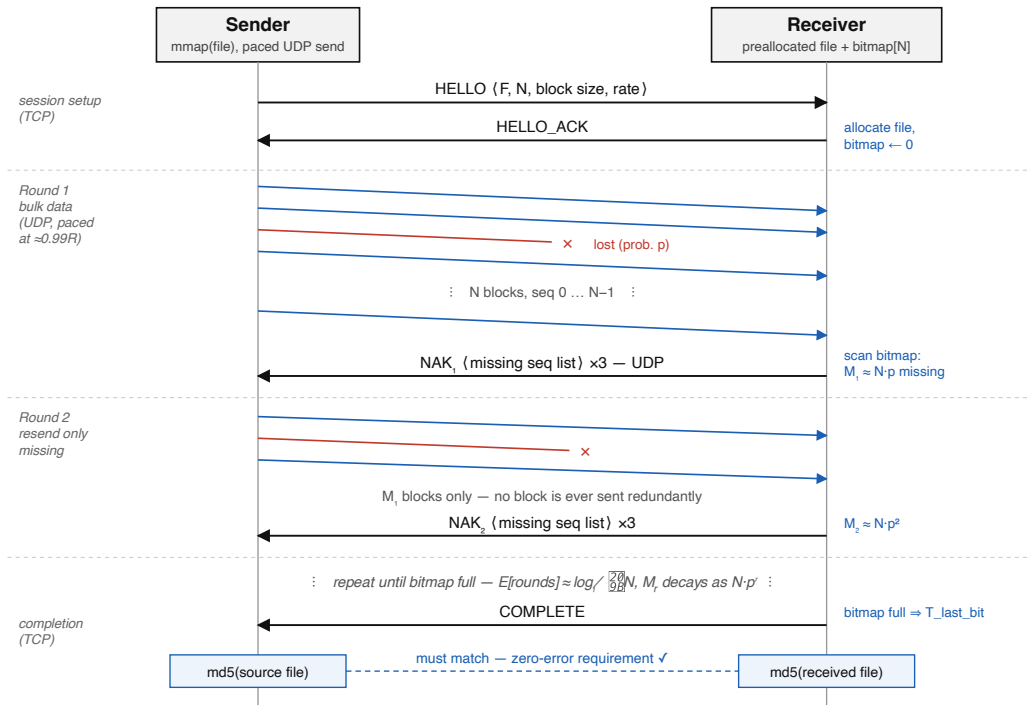


Figure 2: One transfer. TCP carries only session setup and the final confirmation. Data blocks (1464 B payload + 8 B header, filling a 1500 B frame; $N = 733,431$ blocks) flow over UDP, paced at the known rate. After each round the receiver sends its missing-sequence list (NAK, 3 copies over UDP); the sender retransmits exactly those blocks. State is one bitmap (~ 90 KB) on the receiver and an mmap of the file on the sender. The transfer ends only when the bitmap is full, so zero-error delivery is structural, not probabilistic.

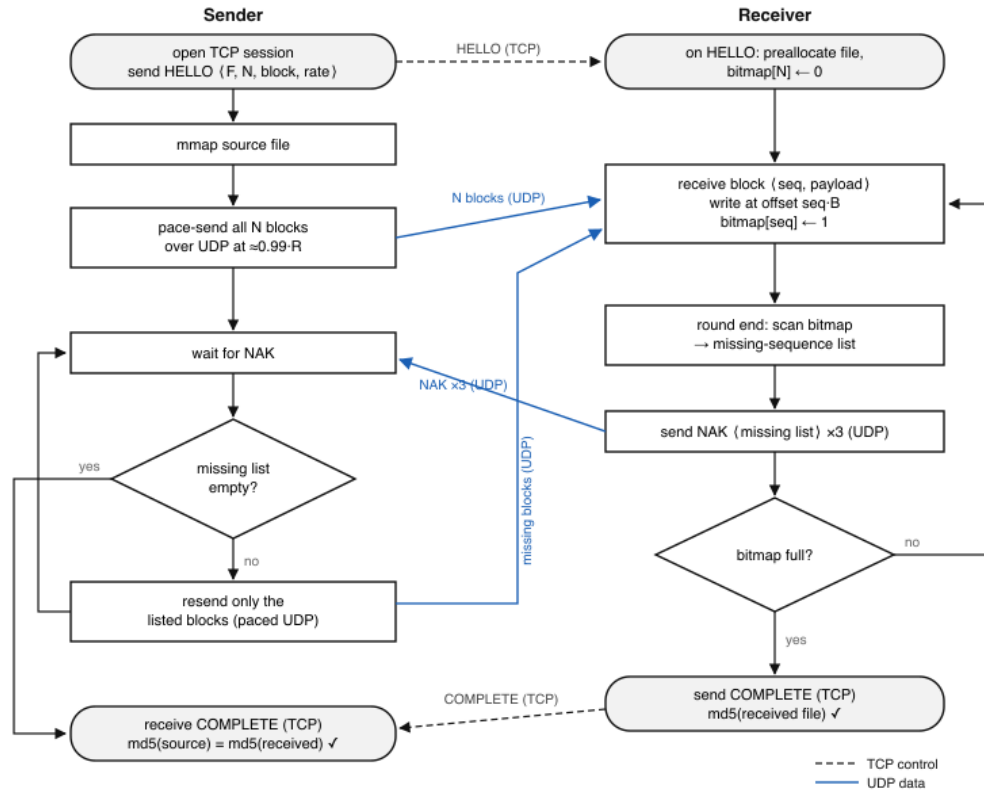


Figure 3: Flow chart of the transfer algorithm. Solid rectangles: processing; diamonds: decisions; dashed black: TCP control; blue: UDP data path. The sender loop retransmits only the blocks named in the latest NAK; the receiver loop ends when the bitmap is full.

4 Measurements

Figure 4 summarizes the measured one-way times; Figure 5 shows how each transfer completes over time; Figure 6 shows the same runs as goodput against the measured UDP delivery ceiling (Figure 1).

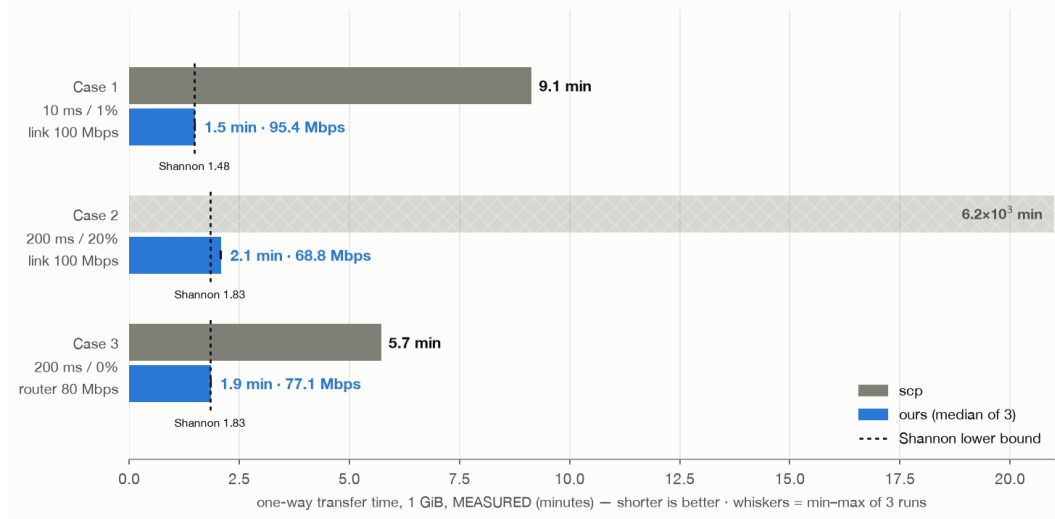


Figure 4: One-way transfer time for 1 GiB (3 reps/case). scp: 548 s / ≈ 4 d / 343 s; ours: 90–125 s in every case.

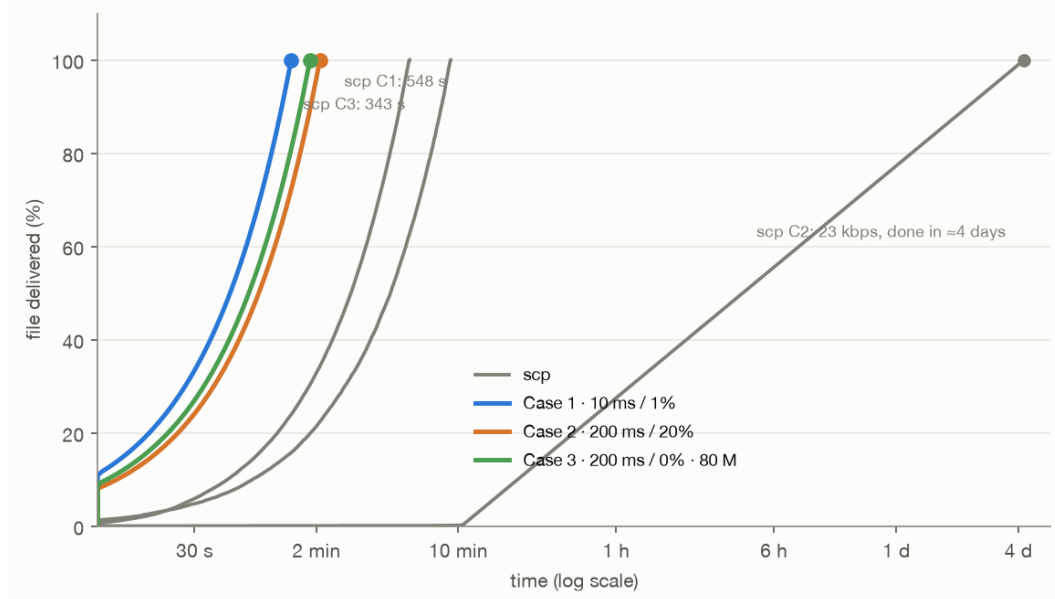


Figure 5: Cumulative delivery on a log time axis. Color: our protocol, one representative run per case (curve reconstructed from per-round logs, scaled to the measured duration); gray: scp per-second progress. In linear time the colored curves are straight lines: the delivery rate is constant end to end, because pacing never slows on loss and every retransmitted packet is a needed one. scp Case 2 crawls at 23 kbps and completes only after ≈ 4 days (§4).

Why scp cannot finish (Case 2). scp’s steady state on this link is 23 kbps, i.e. ≈ 4 days for 1 GiB: $\sim 900\times$ below the competition threshold and $\sim 3000\times$ slower than our protocol. Kernel telemetry (`ss -tin`) shows the mechanism: cwnd stays pinned at 1–2 (ssthresh collapses to 7), so three duplicate ACKs never accumulate, fast retransmit never triggers, and every loss is repaired by timeout; the RTO

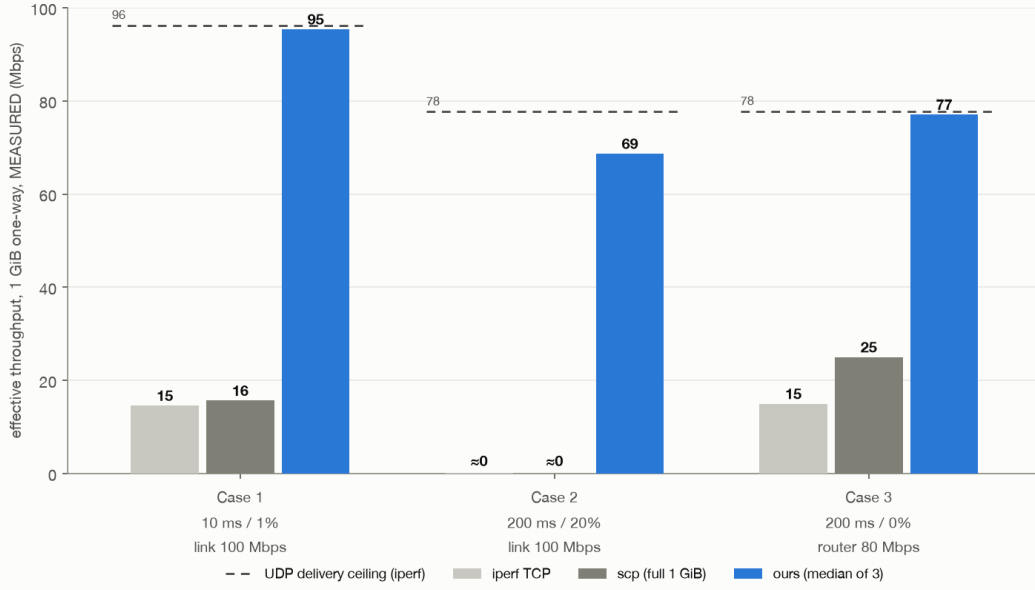


Figure 6: Goodput vs. the measured UDP delivery ceiling (dashed): the protocol operates at the ceiling in all three cases; the remaining gap is retransmission ($= p_{\text{eff}}$) plus header overhead. iperf TCP and scp shown for reference.

backs off exponentially and each repair costs a full timeout. The sender averages 1.9 packets/s, 0.02% of the link’s ~ 8300 packet/s capacity, with $\approx 23\%$ of sent bytes being retransmissions, matching the 20% loss rate. TCP here is not bandwidth-limited: interpreting random loss as congestion, it throttles itself to idleness.

Robustness. A pacing sweep at 85–99 Mbps (Case 2) falls monotonically: sending faster adds tbf-overflow retransmissions, but the higher rate always wins, so the headline runs pace at 99 Mbps ($0.99R$; 79.5 Mbps in Case 3 under its 80 Mbps bottleneck). The testbed adds almost nothing beyond the configured impairment — p_{eff} measured from packet counts is 20.0–20.5% against Case 2’s configured 20%, and within 0.7% of configuration in the other cases — and reliability was never affected: every transfer converged and passed MD5.¹

5 MTU 9001 vs. 1500

Jumbo frames run natively on the same topology, and they are *faster* in every case (Figure 7): the header overhead falls ($\eta = 1464/1500 = 0.976 \rightarrow 8965/9001 = 0.996$) and the same rate needs $6\times$ fewer packets per second. Measured p_{eff} stays at the configured loss for both MTUs, so the specified burst 9015 shaper passes paced jumbo UDP cleanly; the fastest configuration, Case 1 at MTU 9001, reaches 96.8 Mbps — within 1.5% of that link’s erasure bound.

¹ 1 GiB test file MD5 2322915a33d9a0fc4f1dab8a8b795c8f, verified sender- and receiver-side in every reported run.

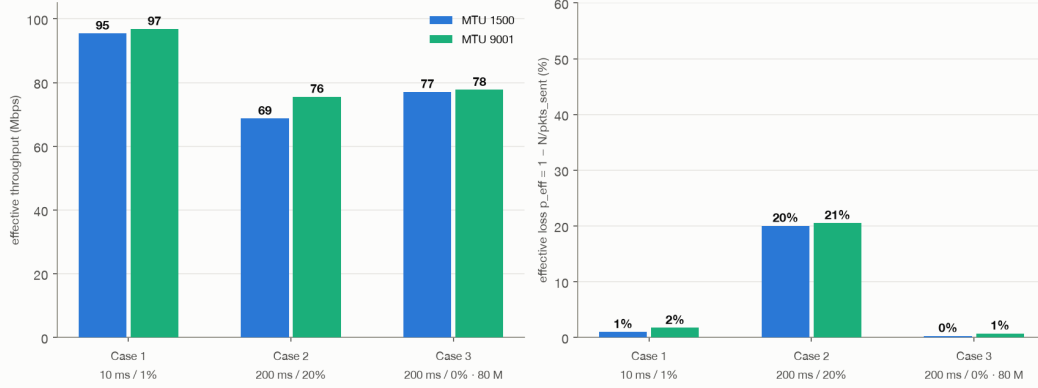


Figure 7: Left: goodput with tuned pacing, median of 3 runs per bar, all MD5-verified — jumbo is faster in all three cases. Right: measured effective loss $p_{\text{eff}} = 1 - N/\text{packets sent}$ matches the configured loss at both MTUs; the shaper adds at most 0.7% on top of netem.

6 Conclusion

On a dedicated link with known parameters, reliability is cheap: pace at line rate, and retransmit only what the bitmap says is missing. This delivers 1 GiB in about two minutes in every case: $3.1\text{--}6.1\times$ faster than scp where scp finishes at all, and within 1–13% of the erasure-channel limit. In the two low-loss cases the gap is essentially just round overhead (+1.2%); Case 2’s larger gap is the extra rounds that 20% bidirectional loss forces (§A). Since the overhead beyond the Shannon bound is only a few seconds, no FEC or further protocol machinery can improve it meaningfully; the remaining gains are environmental (shaper configuration, NAK pacing).

A Analysis: provably near-optimal

We work through Case 2 ($R = 100\text{ Mbps}$, $\text{RTT} = 200\text{ ms}$, $p = 0.20$); Cases 1 and 3 follow from the same expressions.

Setup. The file is $F = 1\text{ GiB} = 2^{30}\text{ B} = 8,589,934,592\text{ bits}$. Each block carries 1464 B of payload in a 1500 B frame, so the payload efficiency is $\eta = 1464/1500 = 0.976$ and the block count is

$$N = \lceil 2^{30}/1464 \rceil = 733,431. \quad (1)$$

Lower bound. A link that drops each frame independently with probability p is an erasure channel of capacity $(1 - p)R$, of which a fraction η is payload. No protocol or code can deliver the file faster than

$$\begin{aligned} T_{\min} &= \frac{F}{(1 - p)\eta R} \\ &= \frac{8,589,934,592\text{ b}}{0.80 \times 0.976 \times 10^8\text{ b/s}} = \frac{8,589,934,592}{7.808 \times 10^7}\text{ s} = 110.0\text{ s}. \end{aligned} \quad (2)$$

The protocol meets the bound in expectation. Each block must be received once. A given transmission survives with probability $1 - p$, and the bitmap guarantees no block is re-sent after it has been received, so the number of transmissions of one block is geometric with mean $1/(1 - p)$ and the total is

$$\mathbb{E}[\text{packets}] = N(1 + p + p^2 + \dots) = \frac{N}{1 - p} = \frac{733,431}{0.80} \approx 916,789. \quad (3)$$

Paced back-to-back at the line rate, these frames occupy the wire for

$$\frac{916,789 \times 1500 \times 8\text{ b}}{10^8\text{ b/s}} = 110.0\text{ s} = T_{\min}, \quad (4)$$

i.e. constant-rate sending plus retransmit-only-what-is-missing wastes no transmission in expectation. The only cost left is the round structure.

Round overhead. Round 1 sends all N blocks; round $k+1$ re-sends the expected Np^k blocks still missing. For Case 2 the expected round sizes, each $0.2\times$ the previous, are

$$733,431 \rightarrow 146,686 \rightarrow 29,337 \rightarrow 5,867 \rightarrow 1,173 \rightarrow 235 \rightarrow 47 \rightarrow 9 \rightarrow 2 \rightarrow <1. \quad (5)$$

The transfer ends when the slowest block has arrived, i.e. after the maximum of N i.i.d. geometric variables, whose expectation is

$$\begin{aligned} \mathbb{E}[\text{rounds}] &\approx \log_{1/p} N + \frac{\gamma}{\ln(1/p)} + \frac{1}{2} \\ &= \frac{\ln 733,431}{\ln 5} + \frac{0.5772}{\ln 5} + 0.5 = 8.39 + 0.36 + 0.50 = 9.3 \end{aligned} \quad (6)$$

(γ is the Euler–Mascheroni constant). Each round boundary costs one RTT of sender idle time (the last block’s propagation plus the NAK’s return), so the expected overhead beyond $T_{\min}$ is

$$\mathbb{E}[\text{rounds}] \cdot \text{RTT} = 9.3 \times 0.2 \text{ s} = 1.9 \text{ s}, \quad 1.7\% \text{ of } T_{\min}. \quad (7)$$

Predicted times. Hence

$$T \approx T_{\min} + \mathbb{E}[\text{rounds}] \cdot \text{RTT} = 110.0 + 1.9 = 111.9 \text{ s} \quad (8)$$

for Case 2; the same computation gives 88.9 s for Case 1 (+0.04%) and 110.2 s for Case 3 (+0.2%). That ~ 2 s is also the ceiling on what any FEC scheme could save, which is why we use none. Cases 1 and 3 land within 1.2% of this prediction (measured 90.0 s and 111.4 s). Case 2’s measured 124.8 s exceeds it because the idealized model assumes every NAK is delivered: under 20% loss in *both* directions the NAK feedback is itself lost, so the transfer takes ≈ 46 rounds rather than the 9.3 the one-way model predicts, and each extra round costs an RTT ($36 \times 0.2 \text{ s} \approx 7 \text{ s}$). Measured p_{eff} equals the configured 20% (Figure 7), confirming the excess is round overhead, not extra loss.